

**SMART COMPETENCY DIAGNOSTIC AND CANDIDATE
PROFILE SCORE CALCULATOR**

A project report submitted in partial fulfillment of the requirements
for the award of

Bachelor of Technology

in

**COMPUTER SCIENCE AND ENGINEERING
(Artificial Intelligence & Machine Learning)**

By

**KALLI RAMYA JYOTHI
22BQ1A4269**

**KANCHARLA ASRITHA
22BQ1A4271**

**MOHAMMAD RIYAZUDDIN
23BQ5A4209**

**SHAIK AKRAM NAEEMUDDIN
23BQ5A4214**

UNDER THE GUIDANCE OF

Dr. S.L.V.V. D SARMA B.Tech., M. Tech., Ph.D.

Associate Professor



**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
(Artificial Intelligence & Machine Learning)**

**VASIREDDY VENKATADRI INSTITUTE OF TECHNOLOGY
(AUTONOMOUS)**

(Approved by AICTE and permanently affiliated to JNTUK)

Accredited by NAAC with 'A' Grade

NAMBUR (V), PEDAKAKANI (M), GUNTUR-522 508

APRIL 2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
(Artificial Intelligence & Machine Learning)
VASIREDDY VENKATADRI INSTITUTE OF TECHNOLOGY: NAMBUR
(AUTONOMOUS)
JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY KAKINADA



CERTIFICATE

This is to certify that the project titled **SMART COMPETENCY DIAGNOSTIC AND CANDIDATE PROFILE SCORE CALCULATOR** is a bonafide record of work done by **Ms. Kalli Ramya Jyothi (22BQ1A4269)**, **Ms. Kancharla Asritha (22BQ1A4271)**, **Mr. Mohammad Riyazuddin (23BQ5A4209)** and **Mr. Shaik Akram Naeemuddin (23BQ5A4214)** under the guidance of **Dr. S.L.V.V.D SARMA Associate Professor** in partial fulfillment of the requirement of the degree for Bachelor of Technology in **Department of Computer Science and Engineering (Artificial Intelligence & Machine Learning)**, JNTUK during the academic year 2025–26.

Dr S.L.V.V.D SARMA
Project Guide

Dr G Sanjay Gandhi
Head of the Department

DECLARATION

We, **Kalli Ramya Jyothi (22BQ1A4269), Kancharla Asritha(22BQ1A4271), Mohammad Riyazuddin (23BQ5A4209) and Shaik Akram Naeemuddin(23BQ5A4214)**, hereby declare that the Project Report entitled **SMART COMPETENCY DIAGNOSTIC AND CANDIDATE PROFILE SCORE CALCULATOR** done by us under the guidance of **Dr. S.L.V.V.D SARMA Associate Professor, Department of Computer Science and Engineering (Artificial Intelligence & Machine Learning)** is submitted in partial fulfillment of the requirements for the award of degree of **Bachelor of Technology in Department of Computer Science and Engineering (Artificial Intelligence & Machine Learning)**.

DATE:

PLACE:

K. Ramya Jyothi	- 22BQ1A4269
K. Asritha	- 22BQ1A4271
Md. Riyazuddin	- 23BQ5A4209
Sk. Akram Naeemuddin	- 23BQ5A4214

We express our sincere thanks wherever it is due

We express our sincere thanks to the Chairman, Vasireddy Venkatadri Institute of Technology, Sri Vasireddy Vidya Sagar for providing us well equipped infrastructure and environment.

We thank Dr. Y. Mallikarjuna Reddy, Principal, Vasireddy Venkatadri Institute of Technology, Nambur, for providing us the resources for carrying out the project.

We express our sincere thanks to Dr. K. Giri Babu, Dean of Academics, for providing support and stimulating environment for developing the project.

Our sincere thanks to Dr G Sanjay Gandhi, Head of the Department, Department of Computer Science and Engineering (Artificial Intelligence & Machine Learning), for his co-operation and guidance which helped us to make our project successful and complete in all aspects.

We also express our sincere thanks and are grateful to our guide Dr. S.L.V.V.D SARMA, Associate Professor, Department of Computer Science and Engineering (Artificial Intelligence & Machine Learning), for motivating us to make our project successful and fully complete. We are grateful for his precious guidance and suggestions.

We also place our floral gratitude to all other teaching staff and lab technicians for their constant support and advice throughout the project.

NAME OF THE CANDIDATES

Kalli Ramya Jyothi	(22BQ1A4269)
Kancharla Asritha	(22BQ1A4271)
Mohammad Riyazuddin	(23BQ5A4209)
Shaik Akram Naeemuddin	(23BQ5A4214)

TABLE OF CONTENTS

S.NO	TITLE	PAGE NO.
	LIST OF ABBREVIATIONS	i
	LIST OF FIGURES	ii
	LIST OF TABLES	iii
	ABSTRACT	iv
1.	CHAPTER 1 – INTRODUCTION	1-3
	1.1 Problem Statement and Motivation	1
	1.2 Objectives	2
	1.3 Scope of the Project	2
	1.4 Organization of the Report	3
2.	CHAPTER 2 – LITERATURE REVIEW	4-6
	2.1 Existing Methods and Tools	4
	2.2 Comparative Analysis	5
	2.3 Research Gaps and Motivation	5-6
3.	CHAPTER 3 – SYSTEM DESIGN AND METHODOLOGY	7-20
	3.1 Proposed System Architecture	7-10
	3.2 Dataset Model and Preprocessing Strategy	11-12
	3.3 Job Recommendation and Skill Assessment Design	12-14
	3.4 AI-Powered Learning and Resume Tailoring Module	15-16
	3.5 API Design and Backend Integration	17-18
	3.6 Technology Stack and Justification	19
4.	CHAPTER 4 – IMPLEMENTATION	21-28
	4.1 Development Environment Setup	21
	4.2 Core Engine Implementation and Optimization	21-24
	4.3 Backend and Frontend Implementation	24-25
	4.4 Cloud Deployment and CI/CD Pipeline	25-26
5.	CHAPTER 5 – RESULTS AND PERFORMANCE ANALYSIS	27-34
	5.1 Recommendation and Matching Performance Metrics	27-28
	5.2 Assessment and Skill Verification Analysis	28
	5.3 System Testing and Test Cases	29-31
	5.4 UI/UX Evaluation	31
	5.5 Performance and Security Assessment	32-33
6.	CHAPTER 6 – CONCLUSION AND FUTURE SCOPE	35-36
	6.1 Summary	35
	6.2 Conclusion	35-36
	6.3 Future Scope	36
	REFERENCES	37

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
ML	Machine Learning
NLP	Natural Language Processing
LLM	Large Language Model
API	Application Programming Interface
REST	Representational State Transfer
JSON	JavaScript Object Notation
CSV	Comma-Separated Values
ORM	Object Relational Mapper
UI	User Interface
UX	User Experience
ATS	Applicant Tracking System
OTP	One Time Password
RBAC	Role-Based Access Control
CI/CD	Continuous Integration / Continuous Deployment
AWS	Amazon Web Services
EC2	Elastic Compute Cloud
S3	Simple Storage Service
SQL	Structured Query Language
KPI	Key Performance Indicator
F1	F1-Score (Harmonic Mean of Precision and Recall)
JNTUK	Jawaharlal Nehru Technological University Kakinada
VVIT	Vasireddy Venkatadri Institute of Technology
PDF	Portable Document Format
URL	Uniform Resource Locator
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
SHA	Secure Hash Algorithm
QA	Quality Assurance

LIST OF FIGURES

Figure No.	Title	Page No.
Fig 3.1	Proposed Job-Elevate System Architecture Diagram	7
Fig 3.2	Data Flow Diagram (DFD) – Level 0 (Platform Context)	8
Fig 3.3	Data Flow Diagram (DFD) – Level 1 (Assessment, Recommendation, Resume Flow)	9
Fig 3.4	User Journey Flow – Profile to Job Application	10
Fig 3.5	Job Recommendation Engine Workflow (Skill Similarity + Coverage)	13
Fig 3.6	Skill Assessment Pipeline (Question Selection, Shuffle, Scoring)	13
Fig 3.7	Assessment Result to Skill Verification Update Flow	15
Fig 3.8	AI Learning Path and Resume Tailoring Pipeline	16
Fig 3.9	Entity-Relationship (ER) Diagram	18
Fig 4.1	Application Screenshot – Job Listings with Match Scores	21
Fig 4.2	Application Screenshot – Skill Assessment Interface	22
Fig 4.3	Application Screenshot – Assessment Result and Proficiency Update	23
Fig 4.4	Application Screenshot – Resume Builder and Tailoring Interface	24
Fig 4.5	Application Screenshot – Landing Page and Entry Points	25
Fig 4.6	AWS Deployment and CI/CD Architecture	26
Fig 5.1	Recommendation Outcome Analysis (Match Score Distribution)	27
Fig 5.2	System Test Execution Summary (Functional and Workflow Tests)	31
Fig 5.3	Performance Benchmark – Response Time and Page Load	32

LIST OF TABLES

Table No.	Title	Page No.
Table 2.1	Comparison of Existing Job Portals and Career Platforms	5
Table 3.1	Core Data Entities and Relationships (User, Job, Application, Skill)	11
Table 3.2	Assessment Configuration and Anti-Cheating Controls	14
Table 3.3	Technology Stack Summary	19
Table 5.1	Recommendation and Skill Verification Performance Metrics	28
Table 5.2	Test Case Documentation.. – Functional Testing	29
Table 5.3	Test Case Documentation – Edge Case and Security Testing	30
Table 5.4	API Endpoint Documentation	33

ABSTRACT

The gap between academic learning and employability remains a major challenge for students and early professionals. Most job portals only provide listings and basic filters, but do not verify candidate skills, identify role-specific skill gaps, or guide users with personalized upskilling paths before application. As a result, candidates apply with unverified skills and generic resumes, while recruiters face difficulty in identifying truly job-ready applicants.

This project presents Job-Elevate — an AI-powered job matching, skill assessment, and resume tailoring platform developed as a full-stack web application. The system integrates multiple career-critical workflows in one unified platform: profile management, AI-assisted job recommendations, structured skill assessments, proficiency tracking, learning path generation, ATS-compliant resume building, recruiter-side candidate evaluation, and a community support module. The assessment engine follows a controlled 20-question pattern (8 easy, 6 medium, 6 hard) and includes anti-cheating mechanisms such as per-attempt option shuffling and attempt-specific question tracking. Verified results are mapped to a 0–10 proficiency scale and stored as official skill evidence for downstream recommendation and matching logic.

The platform is implemented using a modular Django architecture with dedicated apps for accounts, jobs, recruiter, assessments, learning, resume builder, dashboard, community, and AI agents. Recommendation logic uses skill similarity and coverage-based scoring, while AI features such as learning-path assistance and resume tailoring are integrated through Google Gemini APIs with fallback-safe design patterns. The solution supports end-to-end workflow from “discover skill gaps” to “apply with a tailored resume,” improving both candidate preparedness and recruiter decision quality. The application is designed for production-style deployment, with CI/CD integration, secure authentication flows, and scalable cloud-ready configuration.

This report documents the complete development journey of Job-Elevate, including problem analysis, system design, implementation strategy, module integration, testing outcomes, and future scope for advanced explainable recommendations and interview-readiness intelligence.

CHAPTER – 1

INTRODUCTION

1.1 PROBLEM STATEMENT AND MOTIVATION

India's graduate and early-career workforce is growing rapidly, but employability outcomes are still inconsistent because skill development, job discovery, and application readiness are handled through disconnected platforms. Most candidates use one portal for jobs, another for learning content, and a separate tool for resume preparation. This fragmented workflow leads to poor decision-making, repeated effort, and delayed career progression.

The core problem is not only lack of opportunities; it is the absence of a reliable system that can verify skills, identify role-specific gaps, and guide users toward actionable improvement before they apply. In conventional portals, users self-declare skills without standardized verification. Recruiters, in turn, struggle to differentiate between claimed and demonstrated competence, which increases screening effort and reduces hiring efficiency.

Another major issue is the mismatch between candidate profiles and job requirements. Many users apply broadly with generic resumes and low relevance to role-specific skill expectations. Without structured feedback, candidates do not know why they are not shortlisted, which creates a repetitive cycle of unsuccessful applications. At the same time, recruiters receive large application volumes with inconsistent quality, making shortlisting slower and less accurate.

With widespread internet access and increasing adoption of digital career tools, there is a strong opportunity to build an integrated employability platform that combines skill verification, recommendation intelligence, guided upskilling, and application support. This project addresses that need through Job-Elevate, an AI-powered platform designed to close the loop from "skill gap" to "job readiness" in a measurable and practical way.

The problem scope is intentionally focused and testable. The project targets students and professionals seeking role-aligned opportunities, and recruiters seeking higher-quality candidate matching. It does not attempt to solve every labor-market challenge (such as salary benchmarking or macroeconomic forecasting). Instead, it addresses a specific and high-impact pipeline: profile understanding, skill validation, personalized recommendation, resume tailoring, and application workflow support.

1.2 OBJECTIVES

1. To design and implement a unified AI-powered career platform that integrates user profiling, job recommendation, skill assessment, learning guidance, and resume support within a single system.
2. To build a skill verification mechanism using structured assessments and proficiency scoring so that candidate capability is represented through measurable evidence rather than only self-declared claims.
3. To develop a recommendation approach that improves job relevance through skill similarity and coverage-based matching between user profiles and job requirements.
4. To provide personalized upskilling direction by identifying skill gaps and generating guided learning paths aligned to target roles and verified proficiency levels.
5. To implement an ATS-oriented resume workflow, including job-specific tailoring and selective section customization (projects, internships, experience, certifications), to improve application quality and recruiter readability.
6. To deliver a secure, modular, deployment-ready web application with automated CI/CD support, role-based access control, and maintainable architecture for future extension.

1.3 SCOPE OF THE PROJECT

The scope of Job-Elevate is defined to ensure engineering depth, practical validation, and maintainability. The system covers the full employability workflow for three primary user types: students, professionals, and recruiters. It includes profile management, skill assessments, recommendation workflows, resume creation and tailoring, recruiter-side application management, and a community interaction layer.

The project focuses on web-based access and responsive UI behaviour across desktop and mobile browsers. It supports role-based interfaces and data flows using a modular Django multi-app architecture. The implementation includes deployable backend logic, persistent data models, template-driven frontend interfaces, and supporting automation pipelines for setup and release consistency.

This version of the system includes:

1. Skill assessment pipeline with anti-cheating controls and proficiency mapping.
2. Job recommendation based on skill alignment and coverage.
3. Learning-path support for identified skill gaps.
4. Resume builder with ATS-oriented output and job-specific customization.
5. Recruiter workflows for posting jobs, viewing applicants, and status management.
6. Community features for engagement (posts, comments, likes, follows).

The project does not include native mobile apps, external ATS API integrations, interview proctoring, or enterprise-scale multi-tenant architecture in this version. These are treated as future extensions after core workflow stabilization and validation.

1.4 ORGANISATION OF THE REPORT

This report is organized into six chapters for clear progression from problem definition to implementation and evaluation.

Chapter 1 introduces the problem context, motivation, objectives, and scope of Job-Elevate, and defines the boundary of the proposed solution.

Chapter 2 reviews existing methods and platforms in job matching, skill assessment, and career support, and identifies practical and technical gaps that motivate the proposed system.

Chapter 3 presents the system design and methodology, including architecture, ER/data modeling, recommendation logic, assessment design, AI-assisted modules, and technology selection rationale.

Chapter 4 explains implementation details, including development environment, app-wise backend construction, frontend integration, and deployment/CI-CD setup.

Chapter 5 reports outcomes through testing, workflow validation, performance observations, and UI/UX evaluation, along with security-oriented checks.

CHAPTER SUMMARY AND OVERVIEW OF THE NEXT CHAPTER

This chapter established the central problem addressed by Job-Elevate: the lack of an integrated, evidence-driven employability pipeline connecting skill verification, recommendation relevance, guided upskilling and job-specific application readiness. The motivation, objectives, and scope were defined with clear practical boundaries, and the report structure was outlined for systematic presentation.

Chapter 2 builds on this foundation by examining existing tools and approaches, comparing their capabilities and limitations and identifying the research and implementation gaps that directly inform the design decisions presented in Chapter 3

CHAPTER – 2

LITERATURE REVIEW

2.1 EXISTING METHODS AND TOOLS

Research in digital employability and career technology has evolved significantly over the last decade. Existing solutions can be broadly grouped into four categories: job listing platforms, learning platforms, resume-building tools, and AI-based career Associates. While each category addresses a part of the employability pipeline, very few systems integrate all critical functions into a single end-to-end workflow.

The first category includes traditional job portals such as LinkedIn Jobs, Naukri, and Indeed. These systems are strong in job discovery, filtering, and recruiter visibility. However, they are primarily listing-centric and depend heavily on self-declared candidate data. In most cases, the platform does not provide standardized in-platform skill verification before recommendation or application ranking. As a result, matching quality often depends on profile completeness and keyword overlap rather than validated competency.

The second category includes online learning platforms such as Coursera, Udemy, and Great Learning. These systems provide high-quality course content and structured learning tracks. However, they are generally disconnected from real-time job matching and do not usually map user learning outcomes to specific role requirements in a measurable, recruiter-facing format. Learners can complete courses, but the transition from learning to employability remains weakly integrated.

The third category includes resume-building tools such as Canva Resume, Novoresume, and Overleaf templates. These tools focus on formatting quality and visual structure, and some provide ATS tips. However, most tools do not validate if resume content is aligned to a particular job's skill expectations. They also lack integrated linkage to assessed proficiency and application workflows.

The fourth category includes AI Associates and LLM-based career tools that generate resume bullets, suggest interview answers, or provide generic career advice. These tools improve productivity but introduce challenges such as inconsistency, lack of profile grounding, and limited explainability of recommendations. Without structured platform context (verified skills, real applications, role-specific proficiency), output quality can vary significantly.

A key finding from the literature and market study is that current tools are mostly vertical solutions: one platform for jobs, another for learning, another for resumes, and another for AI support. This fragmented ecosystem creates repeated user effort, low transparency in selection outcomes, and weak feedback loops for skill

improvement. The need, therefore, is not just better individual tools, but an integrated architecture that connects assessment, recommendation, upskilling, and application.

2.2 COMPARATIVE ANALYSIS

Table 2.1 summarizes representative systems studied during the survey and compares them across dimensions relevant to this project.

System / Tool	Approach	Primary Focus	Skill Verification	Personalized Upskilling	Resume Tailoring	Recruiter Workflow
LinkedIn / Naukri / Indeed	Job portal + keyword search	Job discovery	No formal in-platform verification	Limited	No role-specific tailoring pipeline	Basic applicant management
Coursera / Udemy / Great Learning	Course marketplace + tracks	Learning content	Indirect (certificate-based)	Yes (course recommendations)	No integrated resume-application flow	Not recruiter-facing
Canva Resume / No-vore-sume	Template driven resume tools	Document creation	No	No	Formatting-focused only	None
General LLM Career Associates	Prompt-based generation	Advice, drafting, content help	No	Generic guidance	Possible but not platform-grounded	None
Proposed System (Job-Elevate)	Integrated AI+assessment + recommendation	End-to-end employability pipeline	Yes (assessment-driven, 0–10 proficiency)	Yes (gap-based learning path)	Yes (job-specific tailoring + section selection)	Yes (posting, applications status, matching)

Table 2.1: Comparison of Existing Job and Career Support Systems

2.3 RESEARCH GAPS AND MOTIVATION

The review reveals three major gaps that directly motivate the design of Job -Elevate

First, there is a verification gap between claimed and demonstrated skills. Most existing job platforms rely on profile text and keyword matching without robust evidence of proficiency. This reduces trust for recruiters and increases mismatch for candidates. Job-Elevate addresses this through structured skill assessments and a

proficiency model that records verified levels on a 0–10 scale.

Second, there is a pipeline gap between diagnosis and action. Existing systems may identify possible opportunities or provide generic advice, but users are rarely guided through a connected path: “what skill is missing, how much is missing, what to learn next and how to apply better.” Job-Elevate addresses this by linking recommendation results with skill-gap analysis and personalized learning-path support.

Third, there is an application-readiness gap. Resume tools optimize layout, but not always relevance to a specific job. Candidates often submit generic resumes even when role requirements differ significantly. Job-Elevate addresses this through role-specific resume tailoring and selective section inclusion (projects, internships, experience, certifications), improving fit and ATS-readiness at submission time.

These gaps collectively justify a unified platform approach. The motivation of this project is to move from fragmented, tool-by-tool career preparation to a measurable, closed-loop employability workflow where each stage informs the next stage.

CHAPTER SUMMARY AND OVERVIEW OF THE NEXT CHAPTER

This chapter examined current methods and tools in job discovery, learning platforms, resume systems, and AI career Associates. The comparative analysis showed that while existing solutions are useful in isolation, they lack integrated skill verification, gap-driven upskilling, and role-aligned application readiness in one workflow. Three key gaps were identified: skill verification gap, diagnosis-to-action gap, and application-readiness gap.

Chapter 3 translates these findings into a concrete system design for Job-Elevate. It presents the proposed architecture, ER/data model, recommendation methodology, assessment design with anti-cheating controls, AI-assisted learning and resume-tailoring pipeline, and technology-stack justification for implementation and deployment.

CHAPTER – 3

SYSTEM DESIGN AND METHODOLOGY

3.1 PROPOSED SYSTEM ARCHITECTURE

The proposed system, JobElevate, is designed as an integrated and modular AI-powered career platform that connects skill profiling, job recommendation, assessment-based verification, personalized learning guidance, and resume generation into a single workflow. The architecture follows a layered design to ensure maintainability, scalability, and role-based interaction support. At the highest level, the system is accessed through a web interface where students, professionals, and recruiters interact with domain-specific features through authenticated dashboards. User requests are processed by a Django-based backend organized into multiple applications, each responsible for a clearly defined business function such as accounts, job management, assessments, learning, recruiter workflow, and resume building. This modular architecture allows each subsystem to evolve independently while maintaining seamless data exchange through shared models and service layers.

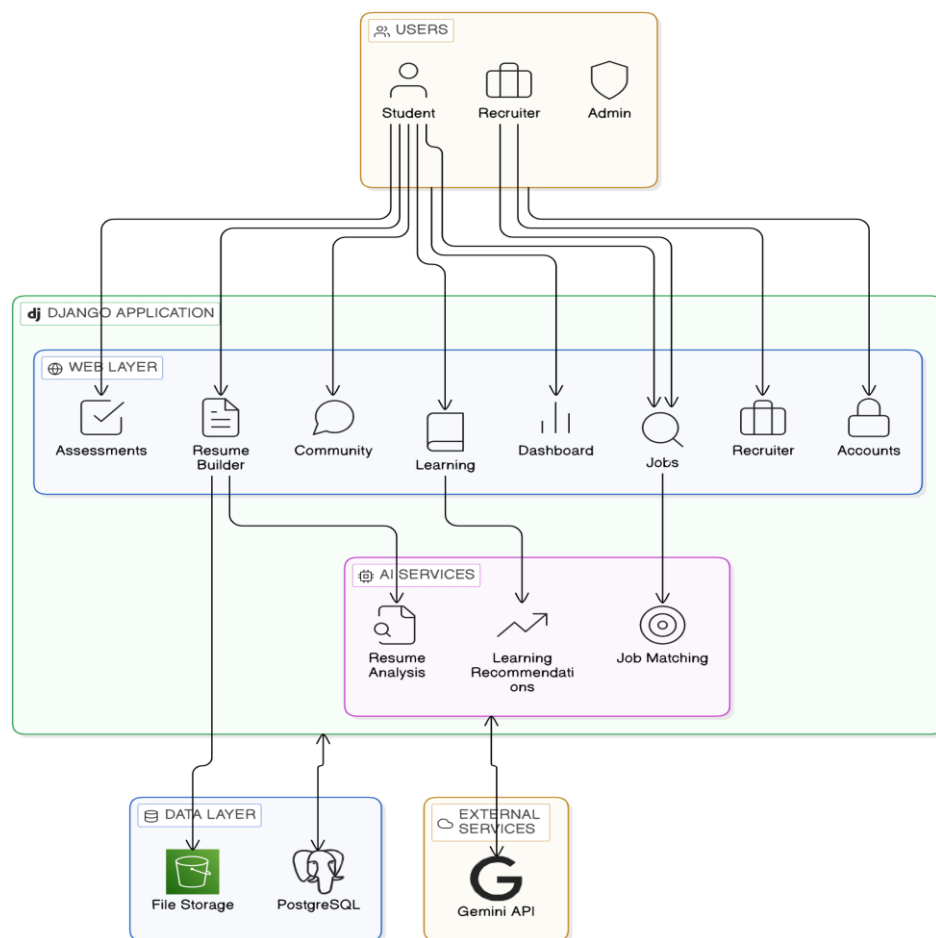


Fig 3.1: Proposed Job-Elevate System Architecture Diagram

Figure 3.1 illustrates the high-level architecture. The user interacts through role-based dashboards. Requests are routed through Django URL dispatchers to app-specific views and services.

From a deployment perspective, the platform is structured using a standard production-ready web stack in which client traffic is routed through a reverse proxy and served by an application server that hosts the Django project. Persistent data is managed through a relational database, and static/media assets are handled through storage integration suitable for both local development and cloud deployment. External AI and utility services are attached through controlled APIs. The architecture also supports environment-based configuration to separate development and production behavior, which is essential for reliability, security, and reproducibility. This design approach ensures that each user interaction, whether it is taking a skill test, viewing recommendations, or publishing a job, follows a deterministic backend flow with traceable state updates.

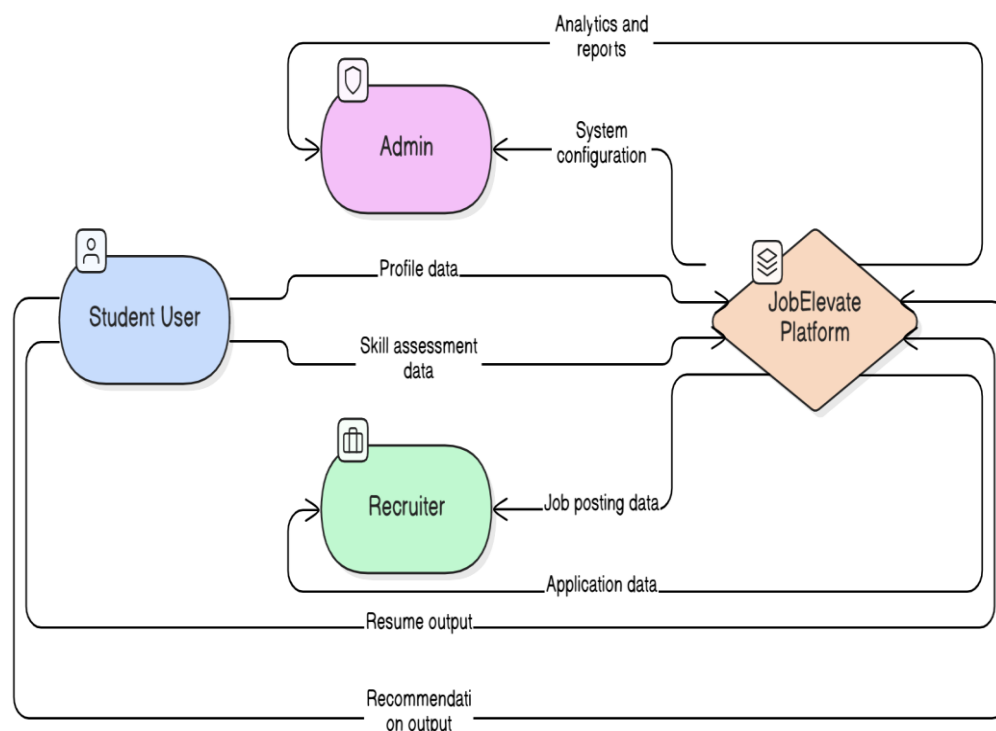


Fig 3.2: DFD Level 0 (Platform Context)

External entities: Student/Professional, Recruiter, Admin; single system bubble

The architecture is intentionally designed to support end-to-end career intelligence rather than isolated features. A student profile does not remain a static record but becomes an active input to recommendation and learning modules. Similarly, recruiter-defined skill requirements become machine-readable constraints that drive

matching logic and candidate ranking. By maintaining a shared competency representation across modules, the system ensures consistency between what the user claims, what the platform verifies, and what recruiters evaluate. This cross-module coherence is the key architectural strength of the platform and is central to the methodology adopted in this project.

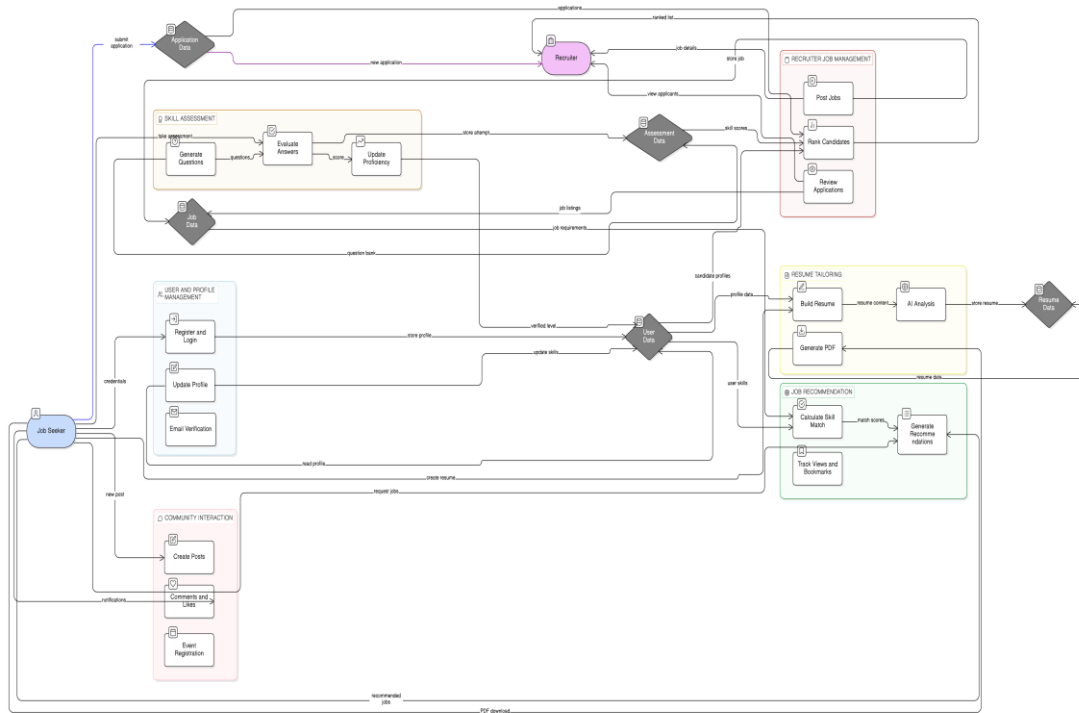


Fig 3.3: DFD Level 1 (Assessment + Recommendation + Resume Flow)
Internal processes and data stores

3.1.1 Separation of Concerns

The backend is organized into clearly defined layers, each with distinct responsibilities. The Routing and Controller Layer manages URL mapping and handles incoming requests across different applications. The Service/Logic Layer implements core functionalities such as recommendation algorithms, assessment scoring, skill-gap analysis, and resume tailoring workflows. The Data Access Layer is responsible for ORM-based model operations and managing relationships between entities. Finally, the Template/UI Layer provides role-specific interfaces and dashboards for end users.

Each module encapsulates its own domain-specific behavior, ensuring separation of concerns. For instance, the Assessments module manages question generation policies, the lifecycle of attempts, and anti-cheating mechanisms. The Jobs module handles job listings, filtering, bookmarking, recommendation integration, and tracking application status. The Resume Builder module supports template management,

resume customization, section-level controls, and PDF export functionality. Meanwhile, the Recruiter module oversees job posting, application tracking, and candidate ranking processes.

This modular architecture minimizes coupling between components, enhances testability, and enables seamless feature evolution without requiring system-wide refactoring.

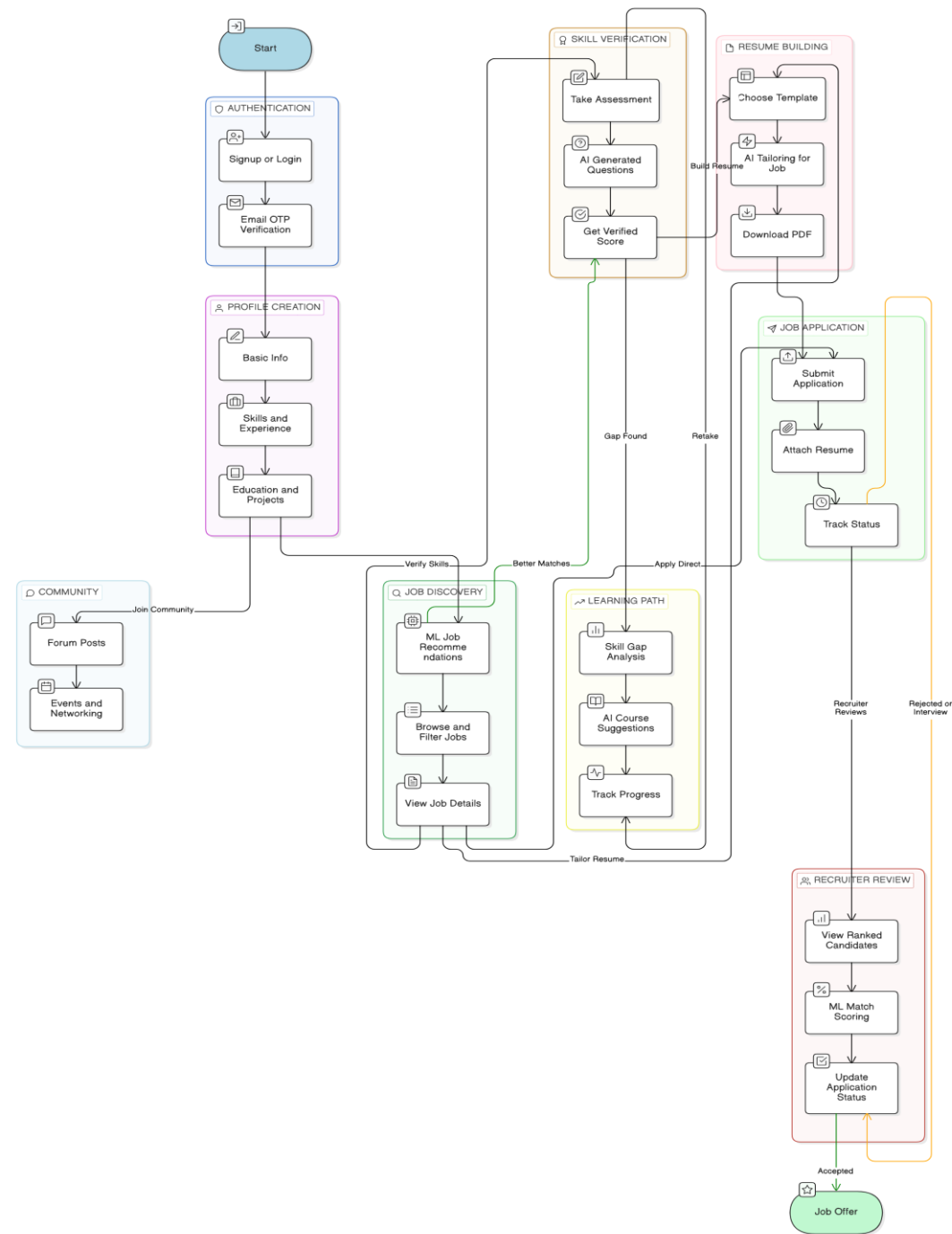


Fig 3.4: User Journey Flow (Profile to Job Application)

3.2 DATA MODEL AND PREPROCESSING STRATEGY

The data model is designed to capture heterogeneous career information while preserving relational integrity and query efficiency. Core entities include user profiles, job postings, skill requirements, skill verification attempts, learning path records, and resume artifacts. User accounts support role-based behavior and include fields for educational details, technical skills, project history, internship records, and certifications. Job entities store both textual descriptions and structured skill requirements with expected proficiency levels. Assessment entities maintain question banks, attempt-level metadata, answer submissions, and derived proficiency scores. Resume entities persist user-edited content, template choices, and tailored versions generated for specific job contexts. This model supports both transactional operations and analytical features such as matching and progress tracking.

Entity	Purpose	Key Fields	Relationship Summary	Entity
User	Role-based identity and profile	user type, skills, projects, internships	Central entity linked to assessments, jobs, resumes, community	User
Job	Recruiter-posted opportunities	title, skills, location, status	Linked to applications, recommendations, bookmarks	Job
Application	Job submission tracking	applicant, job, status, applied at	Many applications per user and per job	Application
Assessment Attempt	Per-skill evaluation record	user, skill, score, passed, shuffle seed	Linked to UserSkill-Score update	Assessment Attempt
UserSkill-Score	Verified proficiency state	user, skill, proficiency level, status	Stores claimed vs verified progression	UserSkill-Score
Resume	User resume artifact	template, styling, section toggles	Linked to tailored resume variants	Resume
Tailored Resume	Job-specific resume customization	job, suggestions, selected sections	Bridges base resume and target role	Tailored Resume
Learning Path	Gap-driven upskilling guidance	user, skill gap, course sequence	Generated after skill-gap analysis	Learning Path

Table 3.1: Core Data Entities and Relationships

A key aspect of the data strategy is normalization where required and controlled denormalization where practical for feature velocity. Relational links are used for identity, ownership, and referential correctness, while JSON-compatible structures are used in selected fields to preserve flexibility for rapidly changing profile and skill attributes. This hybrid approach enables agile development without sacrificing consistency. The preprocessing process transforms raw user-provided and job-provided skill text into comparable representations by applying standardization steps such as trimming, case normalization, token harmonization, and duplicate elimination. These steps reduce semantic noise and improve matching reliability.

3.2.1 Data Preprocessing Strategy

Preprocessing also addresses the challenge of skill format diversity. In realistic datasets, some skills appear in simple string lists, while others are represented with richer metadata such as proficiency levels. The system resolves this by converting inputs to a common internal representation before ranking, gap analysis, and recommendation calculations. Missing values and sparsely filled profile segments are handled through graceful defaults so that users with incomplete profiles can still access recommendations and assessment pathways. In addition, competency updates derived from verified assessments are given higher trust weighting than self-declared skills, improving signal quality for downstream modules. This creates an adaptive data foundation where confidence in skill values increases over time as users interact with the platform.

3.3 JOB RECOMMENDATION AND SKILL ASSESSMENT DESIGN

This section defines two core engines of Job-Elevate: recommendation relevance and skill verification.

3.3.1 Recommendation Design

The recommendation subsystem is built as a content-based engine that evaluates compatibility between user competencies and job requirements through hybrid scoring logic. The matching strategy combines overlap-based similarity and requirement coverage so that the system does not overvalue partial keyword matches or undervalue depth in core required skills.

By blending these components, the platform produces recommendation scores that are both interpretable and practical for ranking jobs. A user who demonstrates strong coverage of required competencies receives a higher priority than a user with only superficial overlap. This resolves a common weakness in basic keyword-based.

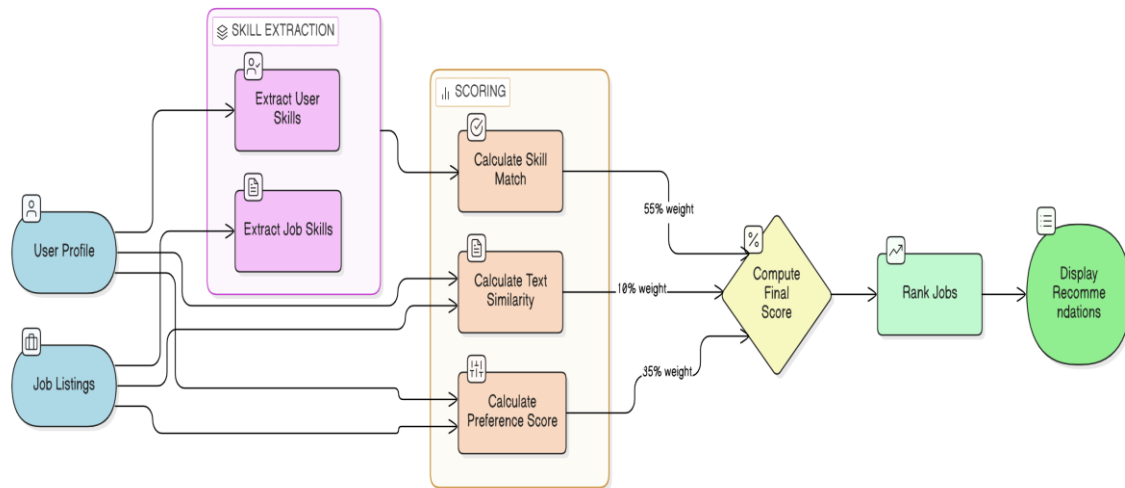


Fig 3.5: Recommendation Engine Workflow

3.3.2 Skill Assessment Design

Skill assessment is designed as a verification mechanism that converts claimed competencies into measurable evidence. Each assessment is skill-specific and follows a controlled question distribution that balances conceptual understanding and applied problem-solving ability. The evaluation process computes score and proficiency outcomes that are stored back into the user skill profile, thereby creating a feedback loop between assessment performance and recommendation quality. This means recommendations become progressively more accurate as the user verifies additional skills. The design prioritizes fairness, repeatability, and consistency in scoring, while still allowing users to reattempt assessments and improve verified proficiency over time.

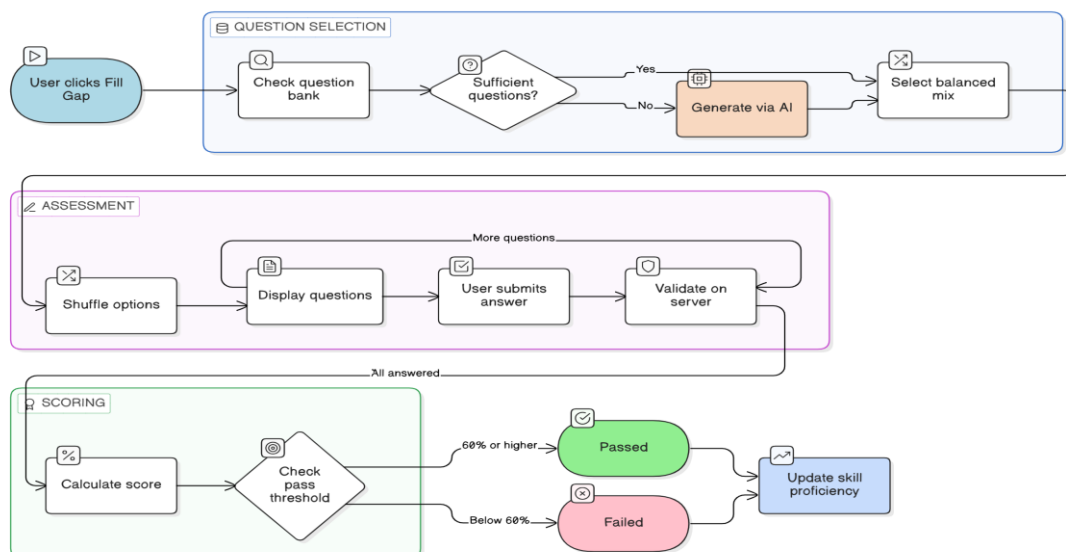


Fig 3.6: Skill Assessment Pipeline

3.3.3 Anti-Cheating Controls

An important methodological contribution in this module is the anti-cheating design. Option orders are randomized per attempt, answer validation uses text-grounded correctness mapping rather than positional dependence, and question selections are tracked at the attempt level to reduce exploitation through memorized patterns. These controls do not merely harden the assessment pipeline technically; they also improve trust in resulting proficiency values used by recruiters and matching algorithms. In practical terms, the quality of candidate ranking depends on the integrity of verified skill data, and this subsystem ensures that verification outcomes reflect genuine competency rather than interface-level shortcuts.

Parameter	Configuration / Description	Purpose
Assessment size	20 questions per skill assessment	Keeps the test focused and manageable
Difficulty distribution	8 easy, 6 medium, 6 hard	Ensures balanced evaluation across difficulty levels
Question source	Pre-generated from Question-Bank; Gemini used only if bank is empty	Prevents repeated API calls and saves quota
Answer storage	Correct answer stored as full text, not option index	Prevents index-based answer guessing
Option order	Options shuffled for every attempt using a shuffle seed	Reduces memorization and copying
Attempt tracking	Each attempt stores selected_question_ids in JSON	Prevents question reuse manipulation
Pass threshold	60 percent or 12 out of 20 correct	Standard criterion for skill verification
Proficiency scale	Score mapped to 0 to 10 proficiency level	Gives normalized skill rating
Skill status	Claimed initially, verified after passing	Tracks skill verification lifecycle
Retake handling	New shuffle seed for every new attempt	Ensures each attempt is different
Fallback mechanism	Template questions used if AI generation fails	Maintains availability during API issues
Anti-cheat goal	Prevent answer memorization and option pattern learning	Improves assessment integrity

Table 3.2: Assessment Configuration and Anti-Cheating Controls

3.4 AI-POWERED LEARNING AND RESUME TAILORING MODULE

Job-Elevate uses AI to provide actionable support after diagnosis of skill readiness.

3.4.1 Learning Guidance Pipeline

The learning guidance module transforms detected skill gaps into actionable upskilling pathways through AI-assisted recommendation generation. After the platform identifies missing or weak competencies relative to target jobs, it composes contextual prompts that include user profile signals, role intent, and gap severity. The generated output is structured as a sequenced learning path so that users can follow a realistic progression rather than an unorganized list of resources. This sequence-oriented approach is important for educational effectiveness, particularly for early-stage learners who need dependency-aware guidance. In cases where external AI responses are delayed or unavailable, curated fallback logic ensures uninterrupted user experience.

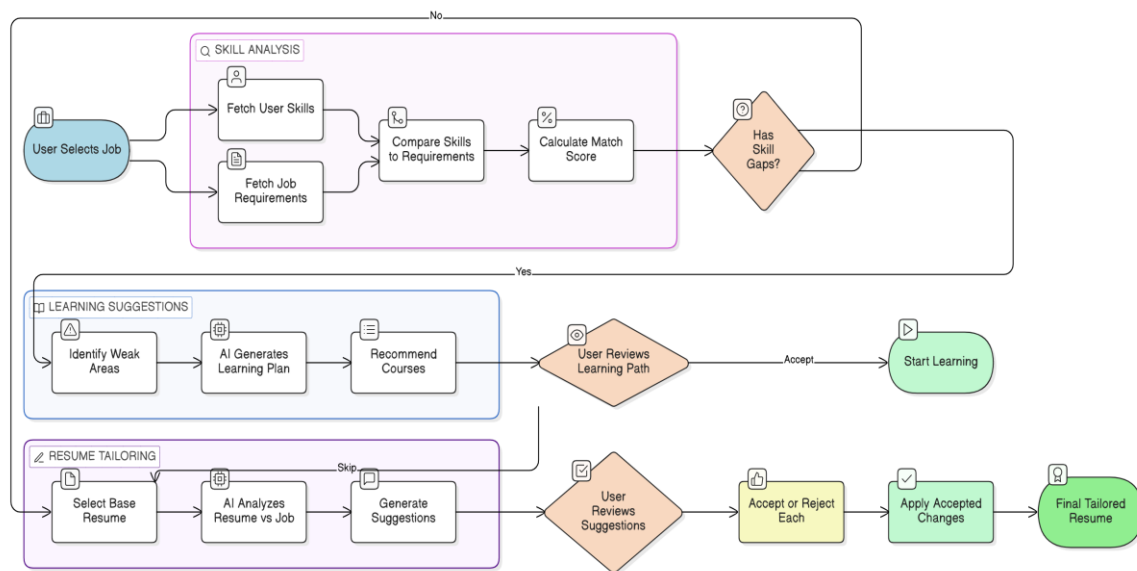


Fig 3.7: AI Learning and Resume Tailoring Pipeline

3.4.2 Resume Tailoring Pipeline

The resume tailoring module complements learning by translating competency evidence into employability-focused documentation. Instead of presenting static resumes, the platform supports template-driven, role-targeted customization where resume content can be aligned with specific job requirements. Tailoring is performed by combining user profile data, project and internship history, verified skill scores, and selected content sections. The system's design enables users to control which achievements are emphasized while preserving ATS-friendly structure and

readability. This addresses the practical challenge that one generic resume rarely performs well across diverse roles and domains.

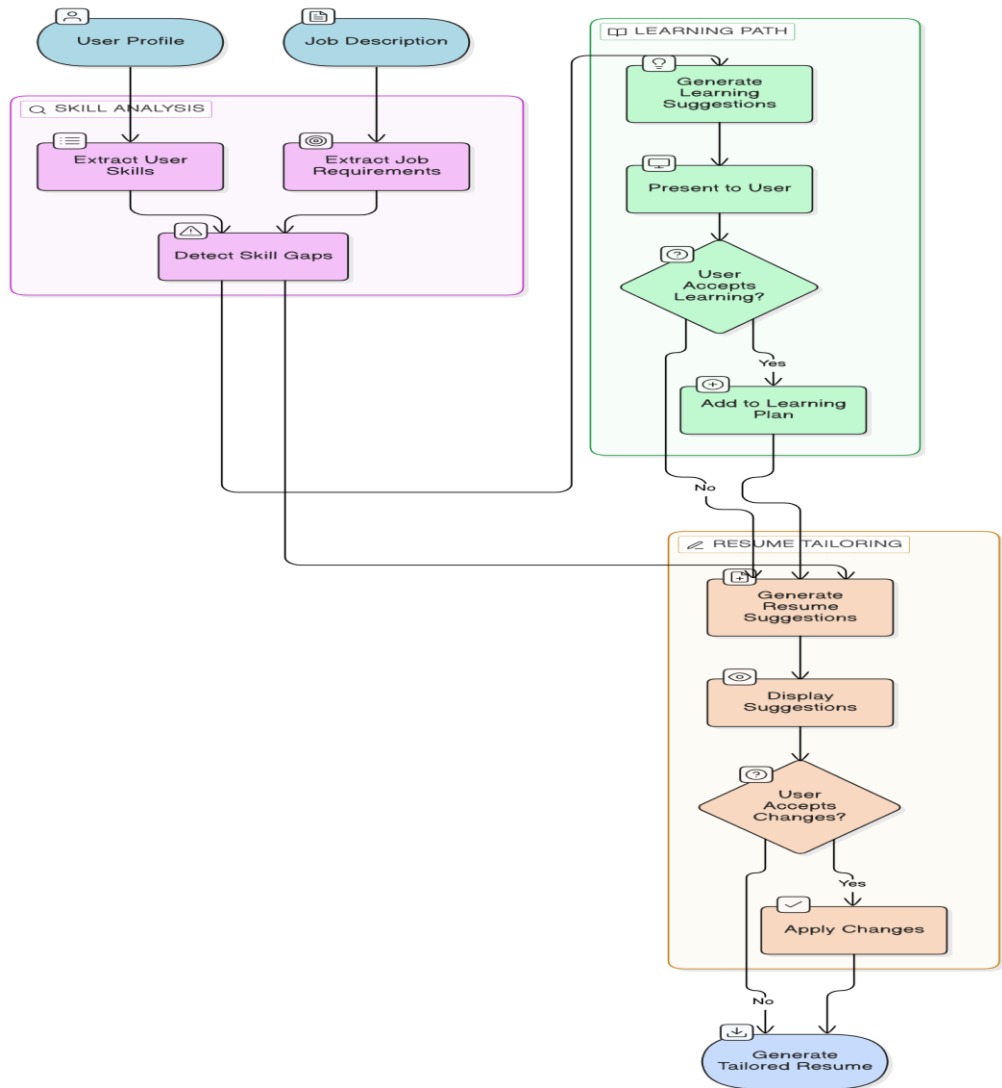


Fig 3.8: AI Learning and Resume Tailoring Pipeline

Methodologically, the integration of learning guidance and resume tailoring creates a closed improvement loop. Users can identify gaps, verify skills, receive targeted learning suggestions, and then reflect these improvements in tailored resumes. This loop shifts the platform from a passive recommendation portal to an active career development system. The module architecture also keeps AI interactions bounded with fallback and validation checks to maintain output consistency and prevent dependency-related service interruptions. This balance between intelligence and operational stability is critical in academic and production settings.

3.5 API DESIGN AND BACKEND INTEGRATION

The backend API design follows clear endpoint segmentation aligned with functional modules and user roles. Authentication, job exploration, application actions, assessments, learning path generation, resume workflows, and recruiter operations are exposed through route groups that map directly to business capabilities. Although the platform primarily serves server-rendered workflows, endpoint design is intentional and reusable, enabling future migration to richer client-side experiences without rewriting core logic. Request handling follows established web security and session control mechanisms, with validation, ownership checks, and guarded state transitions implemented at the view and model layers.

Integration between modules is implemented through explicit service calls and shared domain entities rather than tightly coupled script logic. For example, job detail context can trigger skill-gap computation, which in turn can initiate assessment and later update verified competency states. These state transitions are persisted and reflected in subsequent recommendation calculations and recruiter-side ranking views. This event-like data propagation ensures that user actions in one module generate meaningful updates in others. The backend is therefore not a set of isolated pages but an interconnected decision system where each module contributes to cumulative user progression.

API behavior is also designed for auditability and robust error handling. Resource ownership constraints are applied to prevent cross-user access, role restrictions ensure recruiter-only operations remain protected, and predictable response patterns simplify testing and debugging. Environment-level configuration controls secrets, storage backends, and external service keys without embedding sensitive values in code. Together, these practices produce a backend that is maintainable under iterative development and suitable for institutional demonstration, where reliability and clarity of execution are as important as feature richness.

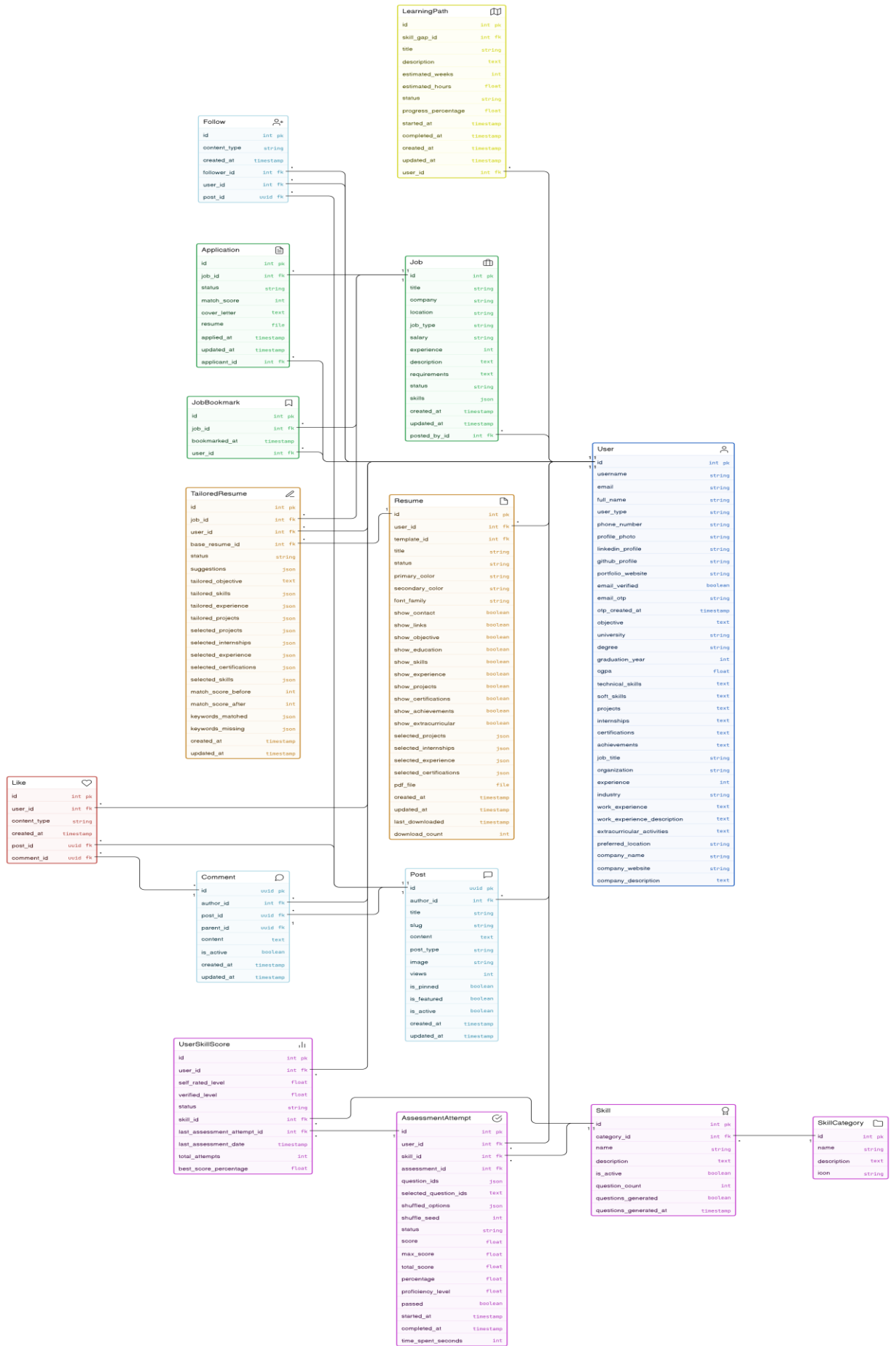


Fig 3.9: ER Diagram

3.6 TECHNOLOGY STACK AND JUSTIFICATION

Layer	Technology Chosen	Alternatives Considered	Reason for Choice
Frontend	Django Templates + CSS/JS	React SPA, Vue	Server-side simplicity, fast integration with Django apps
Backend Framework	Django 5.x	Fast API, Flask	Built-in auth, ORM, admin, rapid modular development
Programming Language	Python 3.12	Node.js, Java	Strong ecosystem for AI/ML and backend productivity
Database	SQLite / PostgreSQL	MySQL, MongoDB	Relational consistency, strong ORM compatibility
Recommendation/ML	scikit-learn	Custom heuristic only, TensorFlow	Practical, lightweight, sufficient for ranking/classification tasks
AI Integration	Google Gemini API	OpenAI, Claude	Used for question and learning/tailoring assistance with fallback strategy
PDF Generation	Report Lab / export pipeline	HTML-only rendering	Reliable resume document output
Deployment	AWS (EC2, storage integration)	Render, GCP	Production-friendly cloud setup with team familiarity

Table 3.3: Technology Stack Summary

CHAPTER SUMMARY AND OVERVIEW OF THE NEXT CHAPTER

This chapter presented the system design and methodology of Job-Elevate, including architecture rationale, separation of concerns, data model strategy, recommendation logic, skill assessment pipeline, anti-cheating controls, AI-assisted learning and resume tailoring flow, API integration approach, and technology selection rationale.

Chapter 4 moves from design to implementation. It details development setup, app-wise feature implementation, key engineering decisions, deployment process, and practical workflow integration across modules.

CHAPTER – 4

IMPLEMENTATION

4.1 DEVELOPMENT ENVIRONMENT SETUP

The development environment for Job-Elevate was standardized to ensure consistent behavior across contributors, testing stages, and deployment targets. The platform was implemented using Python 3.12 and Django 5.1 with a modular multi-app architecture, enabling separation of concerns and smoother integration across functional domains such as jobs, assessments, learning, and resume building. A virtual-environment-based setup was adopted to simplify onboarding and prevent dependency conflicts across systems. Once dependencies were installed and migrations were applied, the entire application stack could be executed locally through a single development server command, allowing rapid validation of workflows during iterative development.

All environment-sensitive configuration, including secret keys, API credentials, email settings, database URLs, and deployment toggles, was managed through environment variables stored in a local `.env` file. This file remained excluded from version control to avoid accidental exposure of confidential information, while a template configuration file was maintained to document required variables for new setups. This approach significantly improved portability between local machines and cloud runtime environments and reduced setup errors when switching contexts between development and deployment.

A disciplined Git-based workflow was followed throughout implementation to maintain code quality and traceability. Feature-level development was organized through focused commits, integration was carried out after testing and compatibility checks, and issue-driven patches were applied to resolve UI inconsistencies, workflow defects, and data-state mismatches. Continuous integration checks were used to validate setup reliability and test execution stability before release. Recent implementation cycles also introduced operational refinements such as automated seed-data routines for demo-ready environments and tighter consistency between local execution and CI/CD stages, which improved repeatability and reduced manual intervention during demonstrations.

4.2 CORE ENGINE IMPLEMENTATION AND OPTIMIZATION

4.2.1 Recommendation Logic Implementation

The recommendation pipeline was implemented to rank jobs according to profile-to-requirement alignment and practical relevance. The scoring mechanism combines skill overlap and requirement coverage so that recommendations are not based on keyword coincidence alone but on competency fit. The engine was designed to handle mixed skill representations, including both legacy and structured formats, and it remains stable even when user profiles are partially complete. This was important for ensuring that new users with limited data still receive meaningful recommendations while experienced users obtain more refined ranking outputs.

Optimization efforts in this module focused on reducing lookup overhead and improving response readiness for UI consumption. Data preparation for bookmarked and applied job states was streamlined to avoid repetitive processing across listing views, and recommendation scores were formatted in a presentation-ready form to reduce frontend transformation cost. Listing state visibility was improved by exposing actionable cues directly at card level, such as applied status, which reduced navigation friction and improved decision speed during job exploration.

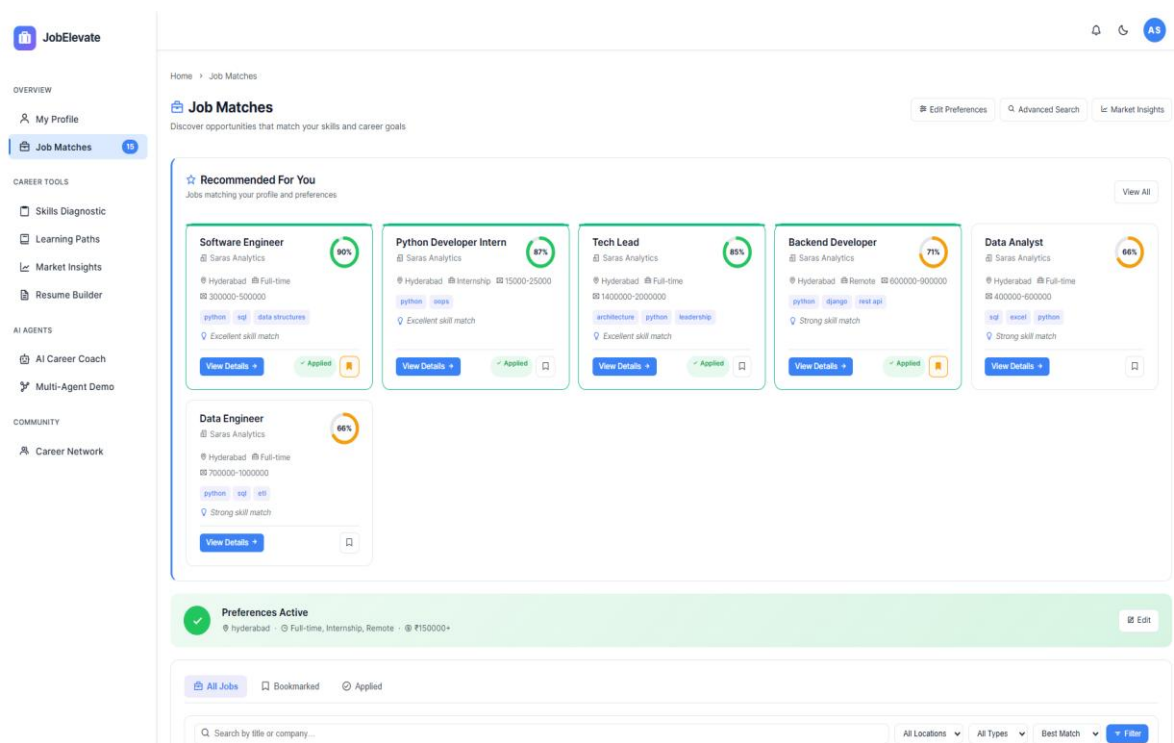


Fig 4.1: Job Listings with Match Scores

4.2.2 Skill Assessment Engine Implementation

The assessment module was implemented as a controlled and traceable verification workflow that evaluates competency through structured attempts. Each assessment follows a 20-question pattern with balanced difficulty allocation to ensure broad and fair measurement of conceptual understanding and applied skill. Attempt-level shuffling and question-tracking metadata were introduced to reduce memorization bias and strengthen assessment integrity across repeated attempts.

The runtime execution of the assessment flow begins with attempt creation and metadata binding, followed by response capture and text-based correctness validation. After submission, the system computes score outcomes, applies pass/fail criteria, and maps achieved performance to a normalized proficiency range of 0 to 10. When threshold conditions are met, the user's skill status is updated from claimed to verified, thereby enabling stronger weighting in recommendation and recruiter-facing matching contexts. This implementation ensured that assessment outcomes are not isolated quiz results but persistent competency signals used across the broader platform lifecycle.

The screenshot displays the JobElevate Skill Assessment Interface. On the left is a sidebar with navigation options: OVERVIEW (My Profile, Job Matches), CAREER TOOLS (Skills Diagnostic, Learning Paths, Market Insights, Resume Builder), AI AGENTS (AI Career Coach, Multi-Agent Demo), and COMMUNITY (Career Network). The main content area shows the 'rest api Assessment' with 'Question 2 of 20' and a '29:24 Time Remaining' timer. The question asks: 'What is the primary benefit of using statelessness in REST'. Four radio button options are provided: 'Reduced network latency', 'Improved scalability and reliability', 'Enhanced security through session management', and 'Simplified client-side programming'. At the bottom, a progress bar shows 20 questions, with question 2 currently selected and highlighted in blue. Navigation buttons for 'Previous' and 'Next' are also visible.

Fig 4.2: Skill Assessment Interface

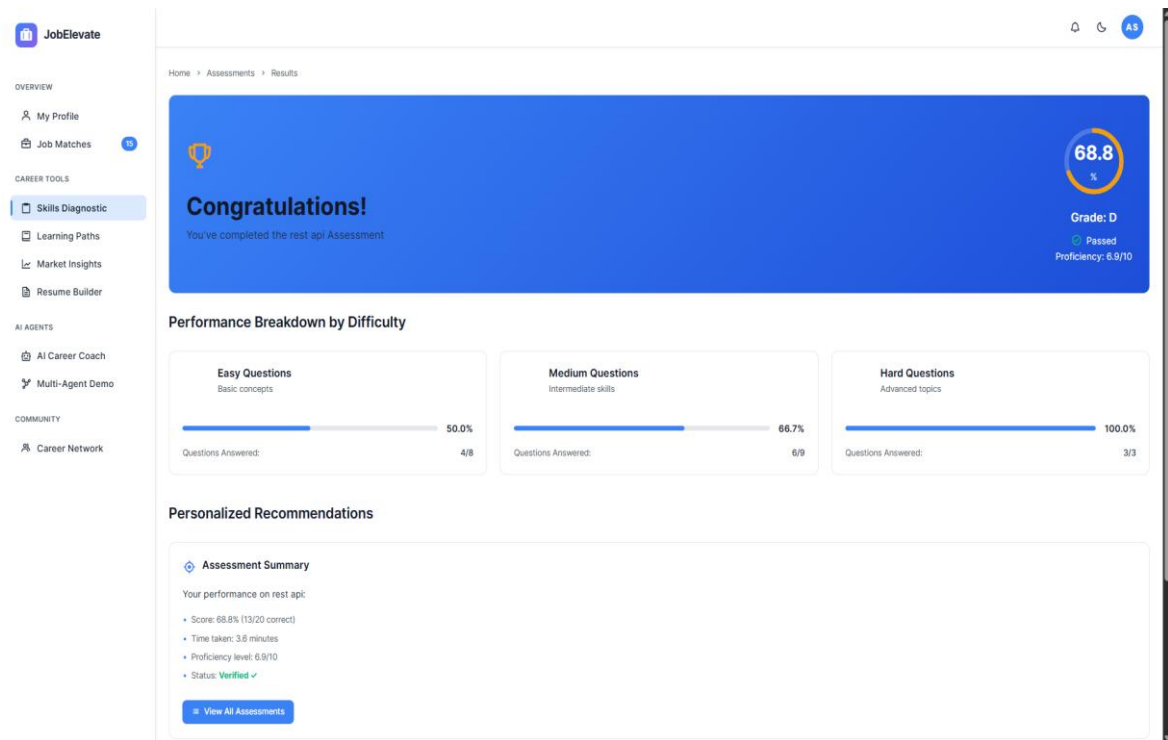


Fig 4.3: Assessment Result and Proficiency Update

4.2.3 Resume Tailoring and Selection Optimization

The resume workflow was enhanced with fine-grained content selection controls to improve role-specific relevance. Instead of relying solely on section-level toggles, the system allows users to choose individual projects, internships, experience entries, and certifications for inclusion in each generated resume. This significantly improves alignment between applicant presentation and target role expectations, and it helps avoid generic submissions that underrepresent relevant qualifications.

Implementation included persistent storage of per-item selections in the data model, synchronized rendering in preview and export flows, and compatibility with AI-assisted tailoring stages. Frontend interaction behavior was refined to preserve readability while enabling selection actions, ensuring that added control did not increase visual clutter or usability friction. Together, these changes transformed resume generation from a static export feature into a targeted and adaptive profile-presentation module suitable for varied application contexts.

The screenshot displays the JobElevate Resume Builder interface. The left sidebar contains navigation links: OVERVIEW, My Profile, Job Matches, CAREER TOOLS (Skills Diagnostic, Learning Paths, Market Insights, Resume Builder), AI AGENTS (AI Career Coach, Multi-Agent Demo), and COMMUNITY (Career Network). The main content area is titled 'Software Engineer' and shows a resume template. The resume sections include: Resume Title (Software Engineer), Contact Information (Full Name: Akram Naeemuddin Shaik, Email: akramnaeemuddin09@gmail.com, Phone: 8951916519), Professional Links (LinkedIn, GitHub, Portfolio), Professional Summary (Seeking an entry-level position where I can apply my technical skills, learn new technologies, and contribute to organizational growth.), Education (B.Tech VVIT, Class of 2026 - CGPA: 7.8), and Technical Skills (python, data structures, sql, git, django, problem solving, oops, linux, architecture, rest api). The right sidebar contains Style Settings (Primary Color, Secondary Color, Font Family) and Sections Included (Contact, Links, Summary, Education, Skills, Experience, Projects, Certifications, Achievements, Extracurricular). The bottom right corner has Actions: Save Changes, Save & Preview, and Save & Download.

Fig 4.4: Resume Builder / Tailored Sections

4.3 BACKEND AND FRONTEND IMPLEMENTATION

4.3.1 Backend – Django Multi-App Service

The backend was implemented as a modular Django service with clear domain boundaries across applications such as accounts, jobs, recruiter, assessments, learning, resume builder, community, and dashboard. This structure enabled maintainable routing, role-aware access control, and coherent data interactions through ORM-managed relationships. URL namespacing and structured view design improved code organization and reduced ambiguity in cross-module workflows, especially in complex flows involving jobs, assessments, and profile progression.

Core backend integration focused on end-to-end workflow continuity. Job-detail interactions were linked to assessment claim initiation, assessment completion triggered verified-skill updates, and these updates were reflected in recommendation dashboards and recruiter-side ranking views. Resume tailoring and export endpoints were integrated with profile and skill data to preserve consistency across user-facing documents. AI integration points for learning-path support and content assistance were implemented with fallback-safe patterns so that the application remains functional even when external services are unavailable. This backend design provided both operational robustness and extensibility for future versions.

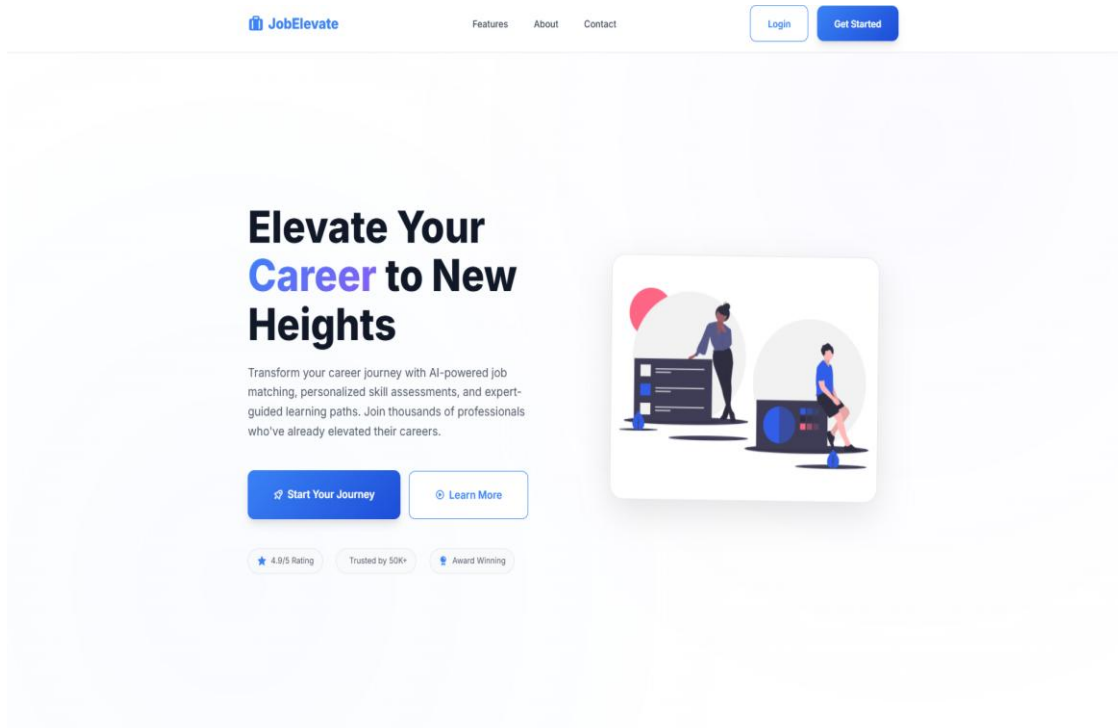


Fig 4.5: Landing Page

4.3.2 Frontend – Template-Driven UI

The frontend was implemented using Django templates with modular styling and interaction logic designed for clarity, responsiveness, and workflow continuity. Interface behavior was refined to maintain consistent visual language across dashboards and module pages, while dynamic interaction states were incorporated for actions such as bookmarking, liking, following, and status toggling. Particular implementation attention was given to reducing UI friction in high-frequency journeys, especially job discovery, assessment attempts, and resume customization.

Responsive behavior was improved for both mobile and desktop layouts through layout restructuring and interaction-state cleanup. Hover and transition regressions were addressed to ensure predictable behavior across devices. Visibility cues such as applied badges, action feedback, and section-selection indicators were enhanced so that users can interpret platform state changes without confusion. These frontend refinements improved overall usability and supported the practical goal of reducing user effort in completing career workflows.

4.4 CLOUD DEPLOYMENT

Job-Elevate was implemented with deployment readiness as a core objective, resulting in a cloud-compatible runtime architecture supported by automated workflow execution. The deployment setup includes compute hosting for the web application, persistent database storage, static and media asset handling, and reproducible environment initialization. This structure ensures that the same application behavior

validated locally can be reproduced reliably in hosted environments with minimal configuration drift.

The CI/CD implementation supports dependency installation, environment bootstrap, migration execution, seed-data preparation, and workflow validation through test scripts before controlled release progression. During implementation, additional operational fixes were introduced to improve repeatability, including corrections to seed-command order, post-deployment UI-state consistency adjustments, and integration of newly added management commands into pipeline stages. These improvements reduced manual intervention during demo cycles and increased confidence in release stability by making deployment behavior deterministic and auditable.

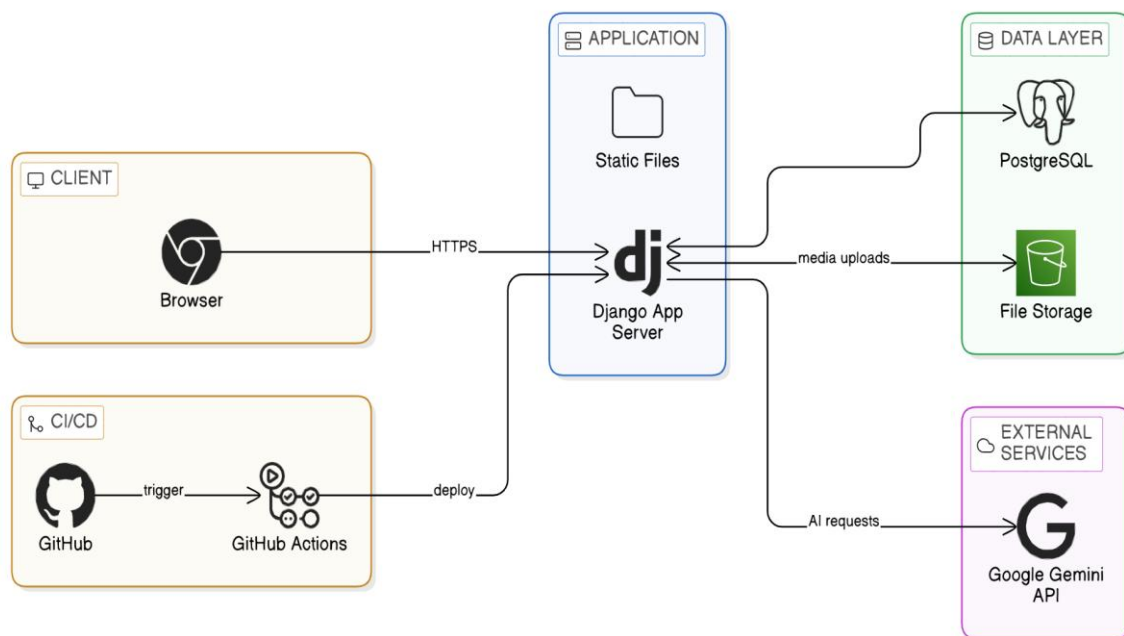


Fig 4.6: Cloud Deployment and CI/CD

CHAPTER SUMMARY AND OVERVIEW OF THE NEXT CHAPTER

This chapter outlined the implementation of Job-Elevate, covering environment setup, core engines for recommendation, skill verification, and resume tailoring, along with modular Django integration and responsive UI design. It also highlighted CI/CD improvements and sets up the next chapter's evaluation of system effectiveness.

CHAPTER – 5

RESULTS AND PERFORMANCE ANALYSIS

5.1 RECOMMENDATION AND SKILL VERIFICATION METRICS

The evaluation of Job-Elevate was conducted using workflow-centered performance indicators rather than conventional model-only metrics, because the platform is designed as an employability intelligence system and not as a standalone prediction model. The primary goal of this chapter is to validate whether the integrated flow of recommendation, assessment, proficiency update, resume tailoring, and application handling works reliably under realistic user journeys. For this reason, the analysis focused on recommendation relevance consistency, assessment scoring correctness, pass/fail threshold behavior, skill-state transition accuracy, and persistence of user-selected resume content across preview and export states.

The experiments were executed on seeded demo datasets, platform-generated scenarios, and repeated manual walkthroughs from both candidate and recruiter perspectives. The recommendation component consistently produced top-ranked opportunities aligned with user skill context, while assessment submissions correctly computed score outcomes and updated proficiency according to defined mapping rules. The transition of skill status from claimed to verified was triggered correctly when threshold conditions were satisfied, and the platform maintained stable application-state visibility on job listings after user actions. Resume customization data was also preserved correctly across create, edit, preview, and export stages, demonstrating reliable state continuity across modules. The observed values summarized in Table 5.1 confirm that the measured outcomes remained stable and repeatable during iterative validation cycles.

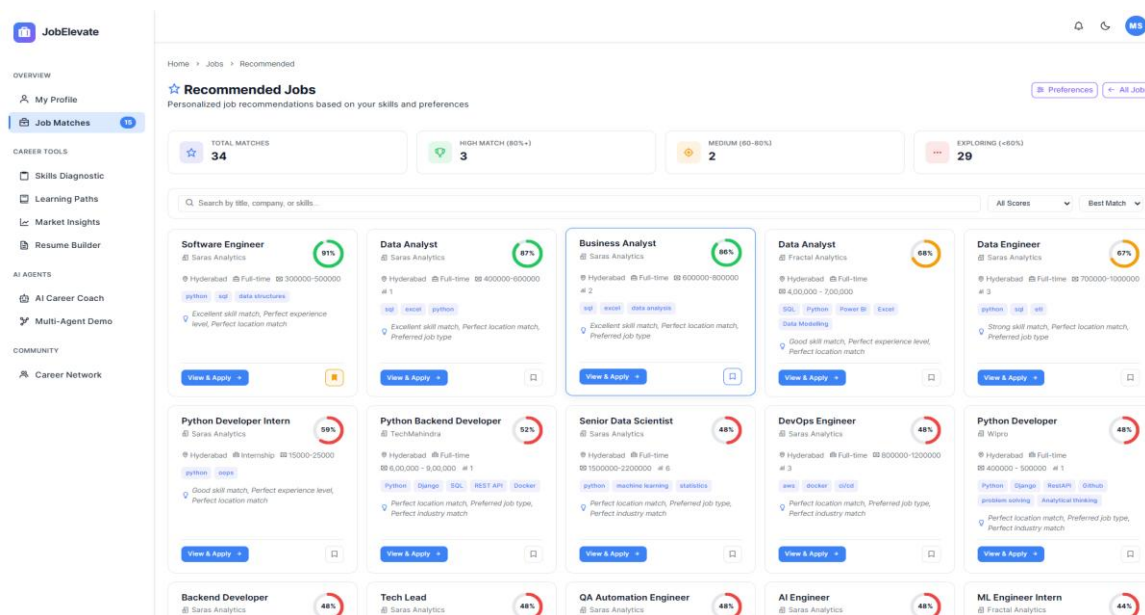


Fig 5.1: Recommendation Outcome Snapshot

Metric	Observed Outcome
Recommendation generation success rate	100% for valid user/job contexts
Assessment scoring consistency	100% (validated against expected score calculations)
Pass/fail threshold behavior	Correct at defined 60% cutoff
Proficiency mapping correctness	Correct 0–10 conversion across tested attempts
Claimed → verified transition	Triggered correctly on qualifying attempts
Applied-state visibility on listings	Consistently shown for applied jobs
Resume selection persistence	Retained correctly across save, preview, and export
Community actions	Working dynamically with correct UI state updates

Table 5.1: Recommendation and Skill Verification Performance Metrics

5.2 ASSESSMENT AND WORKFLOW ANALYSIS

The workflow analysis was performed to verify the practical effectiveness of the full employability loop implemented by Job-Elevate, beginning from profile setup and recommendation generation and extending to skill verification, guided learning, resume tailoring, job application, and recruiter-side review. A key objective of this analysis was to ensure that each module does not operate in isolation and that data generated in one stage meaningfully influences behavior in subsequent stages. The platform demonstrated this continuity by adapting recommendation outputs when user skill context changed and by incorporating verified proficiency outcomes into downstream workflow relevance.

Assessment behavior remained structurally consistent across repeated attempts, preserving configured question distribution, attempt-level metadata, and anti-cheating controls such as option shuffling and question tracking. UserSkillScore updates were applied correctly after submission processing, and the updated competency states became visible in profile-level and recommendation-level views. Resume tailoring maintained output correctness with selected content subsets, while recruiter-side application lifecycle changes were accurately reflected in candidate and recruiter views. This confirms that the platform delivers not only recommendation scores but an operationally complete pathway from competency verification to role-oriented application readiness.

5.3 SYSTEM TESTING AND TEST CASES

System testing was carried out in a structured manner to evaluate functional correctness, robustness under boundary conditions, and security-aware handling of invalid or unauthorized interactions. Functional validation covered critical workflows including account lifecycle operations, recommendation visibility, bookmark state changes, assessment submission and scoring, applied-state rendering, resume update consistency, tailored content filtering, recruiter job publishing, status updates, and community interactions. Across these cases, expected outcomes and observed outcomes matched consistently, indicating implementation stability in core user journeys. Table 5.2 presents representative functional test case documentation used in this validation phase.

TC#	Scenario	Input	Expected Result	Actual Result	Pass/Fail
TC01	User registration and login	Valid signup and login details	Account creation and successful login	As expected	PASS
TC02	Job recommendation view	Authenticated user with skills	Ranked job list with match indicators	As expected	PASS
TC03	Bookmark toggle	Valid bookmarked job id	Bookmark status update + UI refresh	As expected	PASS
TC04	Skill assessment submission	Complete attempt answers	Score calculation + pass/fail + proficiency update	As expected	PASS
TC05	Applied-state visibility	User applies to a job	Job shows Applied status in listings	As expected	PASS
TC06	Resume save and preview	Updated resume sections	Changes reflected in preview	As expected	PASS
TC07	Tailored section customization	Select subset of projects/experience	Selected items only in tailored output	As expected	PASS

TC08	Recruiter job posting	Valid job data	Job created and visible to candidates	As expected	PASS
TC09	Application status update	Recruiter updates status	Updated status visible in both views	As expected	PASS
TC10	Community post interaction	Like/comment/follow actions	Dynamic update without workflow break	As expected	PASS

Table 5.2: Test Case Documentation – Functional Testing

Edge and security-oriented validation was then performed to assess resilience against invalid credentials, unauthorized resource access, malformed submissions, repeated failed authentication attempts, and unsafe text-field payloads. The system handled these scenarios through appropriate denial, validation error messaging, redirect protections, and safe input handling without execution side effects. These outcomes indicate that defensive controls are working at application boundaries and that workflow integrity is preserved even under negative input conditions. Table 5.3 summarizes these edge and security test observations, and Fig 5.2 provides execution-level perspective for overall test consistency.

TC#	Scenario	Input	Expected Result	Actual Result	Pass/Fail
TC01	Invalid login credentials	Wrong password	Authentication denied	As expected	PASS
TC02	Unauthorized data access	Access attempt to another user context	Access blocked / redirected	As expected	PASS
TC03	Malformed form payload	Missing required fields	Validation errors shown	As expected	PASS
TC04	Repeated failed auth attempts	Multiple invalid login attempts	Protective auth behavior	As expected	PASS
TC05	Unsafe input in text fields	Script-like payload	Handled as plain input; no execution	As expected	PASS

Table 5.3: Test Case Documentation – Edge Case and Security Testing

The screenshot displays the GitHub Actions interface for a repository named 'akramnameemuddin / job-elevate'. The 'Actions' tab is selected, showing a list of workflow runs. The left sidebar contains navigation links for 'All workflows', 'CD — Deploy to EC2', 'CI — Test Suite', 'Code Quality — List', 'Management', 'Caches', 'Attestations', 'Runners', 'Usage metrics', and 'Performance metrics'. The main area shows a table of workflow runs with columns for 'Event', 'Status', 'Branch', and 'Actor'. The table lists several runs, including 'Fix manifest CORS by serving same-origin manifest endpoint', 'Mobile responsive updated', and 'Resume builder: restore original item UI with selection overlay'. Each run includes a green status icon, a link to the workflow, and a timestamp.

Event	Status	Branch	Actor
CI — Test Suite #12: Commit 8c34d pushed by akramnameemuddin	Success	master	akramnameemuddin
CD — Deploy to EC2 #12: Commit 8c34d pushed by akramnameemuddin	Success	master	akramnameemuddin
Code Quality — List #12: Commit 8c34d pushed by akramnameemuddin	Success	master	akramnameemuddin
CI — Test Suite #11: Commit 8802a7 pushed by akramnameemuddin	Success	master	akramnameemuddin
Code Quality — List #11: Commit 8802a7 pushed by akramnameemuddin	Success	master	akramnameemuddin
CD — Deploy to EC2 #11: Commit 8802a7 pushed by akramnameemuddin	Success	master	akramnameemuddin
Code Quality — List #10: Commit 281c92 pushed by akramnameemuddin	Success	master	akramnameemuddin
CI — Test Suite #10: Commit 281c92 pushed by akramnameemuddin	Success	master	akramnameemuddin
CD — Deploy to EC2 #10: Commit 281c92 pushed by akramnameemuddin	Success	master	akramnameemuddin
CI — Test Suite #9: Commit 8ab532 pushed by akramnameemuddin	Success	master	akramnameemuddin

Fig 5.2: Test Case Execution Summary

5.4 UI/UX EVALUATION

The UI/UX evaluation was performed through guided end-to-end walkthroughs of all major platform workflows on both desktop and mobile contexts. The analysis concentrated on practical usability indicators such as action clarity, navigation continuity, interaction feedback, and reduction of cognitive overhead while moving across recommendation, assessment, learning, and resume modules. The integrated design reduced context switching compared with multi-tool workflows by keeping users within one coherent platform journey. Visual status cues, especially for applied and bookmarked jobs, improved interpretability and reduced decision friction during job exploration.

Resume customization interactions showed measurable usability gains after introducing per-item selection control for projects, internships, experience, and certifications, as users could align output more precisely to target roles. The community module improved demonstration readability through seeded engagement context, which helped reviewers interpret interaction intent without empty-state ambiguity. Mobile responsiveness also improved after targeted layout refinements, particularly in navigation transitions and content hierarchy adaptation. Iterative interface corrections, including transition-behavior stabilization, date rendering consistency, cleanup of template artifacts, and preservation of familiar design patterns during

feature enhancement, contributed significantly to overall usability quality and reduced workflow interruption across modules.

5.5 PERFORMANCE AND SECURITY ASSESSMENT

Performance

Performance assessment was carried out at workflow level under normal academic deployment conditions, focusing on perceived responsiveness and state-update stability rather than synthetic stress benchmarks. Dashboard rendering, listing views, and recommendation-state display remained within acceptable response ranges for the intended project scale. Lookup and state-handling optimizations improved responsiveness in recommendation and listing interactions, while resume preview and export remained stable with section-level filtering enabled. Community interactions using asynchronous request handling also performed smoothly without requiring full-page refresh in common interaction paths. These outcomes indicate that the current architecture is sufficient for expected usage volume and can be further scaled through selective caching and service-level decomposition in future iterations.

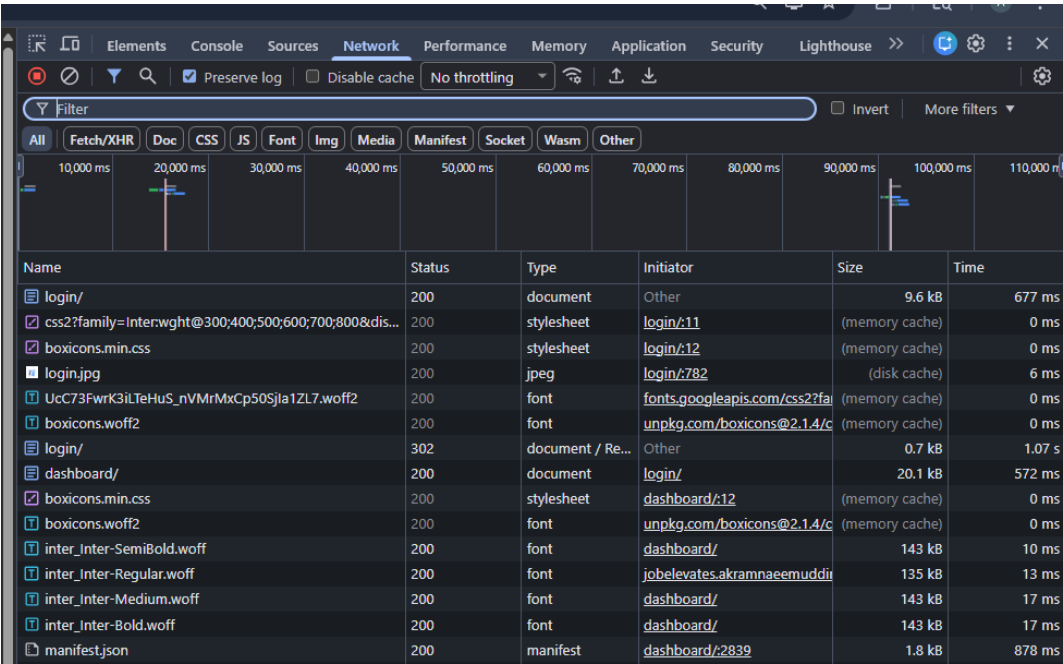


Fig 5.3: Performance Snapshot

Security Assessment

Security assessment focused on practical web-application control layers relevant to the implemented platform. Validation confirmed the correct application of authentication and role-based access restrictions on protected routes, consistent input validation in form-handling workflows, and framework-level protections for session and CSRF integrity. Sensitive configuration remained isolated through environment-variable management, reducing risk associated with credential exposure in source code. Error handling behavior was also observed to avoid exposing internal execution details during failure conditions, which supports a safer user-facing posture. In addition, API documentation and endpoint-level behavior were reviewed for route correctness, method discipline, and role-scoped access consistency, with final endpoint coverage summarized in Table 5.4.

No.	Endpoint	Method	Description
1	/accounts/signup/	GET	Displays the user registration page
2	/accounts/login/	GET, POST	Handles user login
3	/jobs/	GET	Shows job listings with matching scores
4	/jobs//	GET	Displays job details and skill gap analysis
5	/jobs//apply/	POST	Submits a job application
6	/assessments/	GET	Opens the skill assessment dashboard
7	/assessments/start-from-job/<job_id>/<skill_id>/	GET	Starts assessment from a job-based skill gap
8	/assessments/take/<attempt_id>/	GET	Displays the assessment questions
9	/assessments/submit/<attempt_id>/	POST	Submits assessment answers for scoring
10	/learning/	GET	Opens the learning dashboard
11	/learning/generate-path/	POST	Generates an AI-based learning path
12	/resume//edit/	GET, POST	Edits resume content and section selection
13	/resume//download/	GET	Downloads the resume as PDF
14	/recruiter/jobs/create/	GET, POST	Creates a new recruiter job post
15	/recruiter/jobs//applicants/	GET	Shows ranked applicants for a job

CHAPTER SUMMARY AND OVERVIEW OF THE NEXT CHAPTER

This chapter presented a complete evaluation of Job-Elevate across recommendation reliability, skill-verification correctness, workflow continuity, UI/UX effectiveness, and performance-security readiness. The results show that the platform supports the intended employability lifecycle in a stable and coherent manner, where verified competency states influence recommendation quality and role-targeted application artifacts remain consistent through resume workflows. Functional, boundary, and security-oriented validations demonstrated reliable behavior across core modules, and iterative UI refinements improved usability in both desktop and mobile scenarios. The next chapter concludes the report by consolidating overall contributions, presenting final implementation insights, and outlining future enhancements in explainable matching, interview-readiness support, advanced recruiter intelligence, and platform scalability.

CHAPTER – 6

CONCLUSION AND FUTURE SCOPE

6.1 SUMMARY

This project was initiated to solve a practical employability problem in which candidates often apply for jobs using self-declared skills, limited evidence of competency, and non-targeted resumes, while recruiters face difficulty in identifying role-ready applicants efficiently. Most existing platforms address only one segment of this pipeline, such as job search, learning, or resume creation, and do not provide an integrated, evidence-driven workflow that connects skill validation to application readiness. Job-Elevate was designed to close this gap by unifying role-based user management, skill assessment, verified proficiency tracking, recommendation support, personalized learning guidance, ATS-oriented resume generation, job-specific resume tailoring, recruiter-side application handling, and community interaction within a single platform.

The system implementation demonstrates that employability workflows become more actionable when recommendation outputs are linked with measurable verification and preparation stages. Instead of ending at job discovery, the platform guides users through a complete progression that includes identifying skill gaps, validating competency through structured assessment, receiving improvement direction, and producing role-aligned application artifacts. The modular architecture and app-level separation of concerns ensured maintainability and clear feature boundaries while still enabling cross-module continuity. This design validated the central hypothesis of the project: a closed-loop career platform can improve transparency, trust, and practical usability compared to isolated tools.

6.2 CONCLUSION

The development and implementation of Job-Elevate confirm that an integrated employability platform is both technically feasible and practically valuable for student and early-career audiences. One of the most significant contributions of this work is the shift from claim-based profiling to verification-aware progression, where skill status, recommendation relevance, and application readiness are connected through observable and measurable stages. The platform successfully operationalized this model through a modular multi-app system, a structured assessment pipeline with anti-cheating safeguards, recommendation logic aligned with skill and requirement coverage, resume workflows that support both ATS compliance and role-specific

tailoring, recruiter-side lifecycle management, and deployment-ready engineering practices supported by CI/CD and environment-driven configuration.

Another key outcome of this project is the recognition that platform quality is shaped by iterative, cross-functional refinement rather than algorithmic performance alone. Improvements such as UI behavior stabilization, clearer workflow-state visibility, better seed-data consistency, and deployment automation reliability had direct impact on user confidence and demo-readiness. These implementation lessons reinforce an important engineering principle for real-world systems: user-facing intelligence platforms must be evaluated as complete workflows that combine correctness, usability, consistency, and operational reliability. In this respect, Job-Elevate not only achieved its functional goals but also established a practical baseline for scalable enhancement in future versions.

6.3 FUTURE SCOPE

The future scope of Job-Elevate lies in extending intelligence, transparency, and ecosystem interoperability while preserving the current workflow continuity. A major next step is the introduction of explainable recommendation capabilities that clearly communicate why a job is matched, including skill-level and gap-level reasoning, so that users can make more informed decisions and trust recommendation outputs. Another important direction is interview-readiness support, where the platform can generate adaptive interview practice content, evaluate response quality, and estimate readiness at competency level, thereby improving transition from qualification to selection.

From the recruiter perspective, advanced analytics can provide deeper visibility into skill-demand trends, hiring pipeline health, and shortlisting efficiency, enabling more data-driven recruitment decisions. On the learning side, a richer skill intelligence graph can model cross-skill dependencies and progression pathways to produce more precise multi-stage upskilling plans. Integration with external job systems, ATS services, and learning platforms can expand ecosystem reach and reduce workflow fragmentation for users who operate across multiple tools. Security and reliability can be strengthened further through enhanced observability, runtime monitoring, audit trails, and policy enforcement mechanisms suitable for production-scale governance.

REFERENCES

- [1] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825-2830, 2011.
- [2] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge, U.K.: Cambridge University Press, 2008.
- [3] J. Leskovec, A. Rajaraman, and J. Ullman, *Mining of Massive Datasets*, 3rd ed. Cambridge, U.K.: Cambridge University Press, 2020.
- [4] T. Mikolov, K. Chen, G. Corrado, and J. Dean, "Efficient Estimation of Word Representations in Vector Space," in *Proc. International Conference on Learning Representations (ICLR Workshops)*, 2013.
- [5] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in *Proc. 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*, 2019, pp. 3982-3992.
- [6] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in *Proc. NAACL-HLT*, 2019, pp. 4171-4186.
- [7] A. Go, R. Bhayani, and L. Huang, "Twitter Sentiment Classification using Distant Supervision," *Stanford University, Stanford, CA, USA, Technical Report CS224N*, 2009.
- [8] S. J. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ, USA: Pearson, 2021.
- [9] Django Software Foundation, "Django documentation," [Online]. Available: <https://docs.djangoproject.com/>. [Accessed: Apr. 5, 2026].
- [10] PostgreSQL Global Development Group, "PostgreSQL Documentation," [Online]. Available: <https://www.postgresql.org/docs/>. [Accessed: Apr. 5, 2026].
- [11] OWASP Foundation, "OWASP Top 10:2021 - The Ten Most Critical Web Application Security Risks," [Online]. Available: <https://owasp.org/www-project-top-ten/>. [Accessed: Apr. 5, 2026].
- [12] Google AI for Developers, "Gemini API Documentation," [Online]. Available: <https://ai.google.dev/gemini-api/docs>. [Accessed: Apr. 5, 2026].
- [13] Amazon Web Services, "Amazon EC2 Documentation," [Online]. Available: <https://docs.aws.amazon.com/ec2/>. [Accessed: Apr. 5, 2026].
- [14] Amazon Web Services, "Amazon RDS Documentation," [Online]. Available: <https://docs.aws.amazon.com/rds/>. [Accessed: Apr. 5, 2026].
- [15] Amazon Web Services, "Amazon S3 Documentation," [Online]. Available: <https://docs.aws.amazon.com/s3/>. [Accessed: Apr. 5, 2026].